



TEMA II

DESARROLLO DE APLICACIONES BACK EN PHP

*"Siempre parece imposible,
hasta que se hace"*
Nelson Mandela.

7. PROGRAMACIÓN ORIENTADA A OBJETOS

PHP no es un lenguaje orientado a objetos, pero implementa las características que permiten definir las clases.

7.1 Clases y objetos

Una clase define los datos y la lógica de un objeto. La lógica se divide en funciones (métodos) y variables (propiedades).

Para definir una propiedad	<pre>class Coche { public \$color; }</pre>
Para definir un método	<pre>class Coche { public function getColor() { // Contenido de la función } }</pre>

Para crear un objeto hay que instanciar una clase

```
class Coche {  
    // Contenido de la clase  
}  
  
$miCoche = new Coche(); // Objeto
```

Para acceder a un método de la clase se utiliza el operador ->

```
//...  
echo 'Color del coche: ' . $miCoche->getColor();
```

7.2 Ámbito de las propiedades

Los ámbitos de las variables de una clase: private, public, protected

Cuando se define una propiedad como **private**, se indica que ésta no puede verse o modificarse a no ser que sea desde la propia clase. Si utilizamos `$miCoche->color` para verla o definirla, nos dará error. Si queremos definirla, se utiliza un **setter**, si queremos leerla se utiliza un **getter**

```
class Coche {  
    private $color;  
  
    public function getColor(){  
        return $this->color;  
    }  
}  
  
$miCoche = new Coche(); // Objeto  
echo $miCoche->getColor();
```

El operador \$this se utiliza para acceder dentro de una clase a sus propiedades

7.3 Construir un objeto

Los constructores no se sobrecargan, cada clase tiene un único constructor y se utiliza en lugar de la sobrecarga, el paso de parámetros por defecto.

function __construct

7.4 Propiedades y métodos estáticos

Las propiedades y métodos estáticos permiten que sean accesibles sin la necesidad de instanciar la clase. Es importante saber que no se puede acceder a una propiedad static con un objeto instanciado, aunque sí a un método static. El operador de acceso desde fuera de la clase es **::**. Dentro de la clase para acceder a la variable **self::**

```
class Coche {  
    private static $color = 'rojo';  
    private static numeroCoches=0;  
  
    public function mostrarColor()  
    {  
        return self::$color;  
    }  
    public static function getNumeroCoches{  
        return self::numeroCoches;  
    }  
}  
  
$miCoche = new Coche();  
print $miCoche->mostrarColor() ;  
echo Coche::getNumeroCoches();
```

7.5 Herencia

Para indicar que una clase hereda de otra **extends**

Para acceder desde un hijo a su padre **parent::**

El operador **instanceof** funciona igual que en java

7.5 Métodos mágicos

Los métodos mágicos permiten reaccionar a eventos que le puedan ocurrir a un objeto instanciado. Se inician con una doble barra baja, algunos de ellos: **__construct()**, **__destruct()**, **call()**, **__get()**, **__set()**, **__isset()**, **__toString()**.

El más utilizado es el constructor **__construct()**, que se ejecuta cuando el objeto se crea, pudiendo inyectar parámetros y dependencias requeridos para crear el objeto.

El método **__destruct()** hace lo contrario, se ejecuta cuando un objeto se destruye, aunque apenas se utiliza.

Los métodos mágicos **__get()** y **__set()** actúan como **getters** y **setters**

El método **__isset()** actúa como la función **isset()** normal. Este método mágico permite comprobar la existencia de una propiedad no pública.

El método mágico **__toString()** permite devolver el objeto en forma de string.